

Product photo: save as cover.png

DATA SHEET

MagneticTurk

Vision-guided chess-playing robotic arm

A robotic arm that plays chess on a real board. A camera on its wrist reads the position, Stockfish chooses the reply, and the arm picks up and places its own pieces, then checks with the camera that the move landed correctly.

TECHNOLOGIES

Computer vision (OpenCV)
Inverse kinematics
Embedded Linux (Raspberry Pi)
I^2C servo control
Python, Stockfish, web interface

6
servos driving five joints and a gripper

64
squares read by the camera every turn

1
Raspberry Pi running vision, chess and motion

Frank Clarke

October 2026

Contents

1 Overview . . . **3**
1.1 Features . . . 3
1.2 Applications . . . 3
1.3 Key Figures . . . 3
2 How a Move Is Played . . . **3**
3 System Architecture . . . **4**
3.1 The Four Subsystems . . . 4
4 Feature Description . . . **5**
4.1 Active Vision . . . 5
4.2 Reading the Board . . . 5
4.3 Detecting the Player's Move . . . 5
4.4 Choosing a Reply . . . 5
4.5 Motion Planning . . . 5
4.6 Visual Grasp Alignment . . . 5
4.7 Verification and Recovery . . . 5
4.8 Companion Display . . . 5
4.9 Chess Clock . . . 6
5 Chess Rule Support . . . **6**
6 Hardware . . . **6**
7 Specifications . . . **6**
8 Power . . . **7**
9 Software . . . **7**
10 Setup . . . **7**
11 Safety . . . **7**
12 Design History . . . **7**
12.1 Version 1: Magnetic Board . . . 7
12.2 Version 2: Robotic Arm . . . 7
12.3 Hall-Sensor Board . . . 7
12.4 Development Approach . . . 7
13 Acknowledgements . . . **7**

1 Overview

MagneticTurk is a robotic chess opponent that plays on a real board with ordinary pieces. A six-servo robotic arm with a camera on its wrist reads the position, chooses a reply with the Stockfish chess engine, and physically picks up and places its pieces, just as a human player would.

After every move, the robot checks with the camera that the board matches the game before handing the turn back. All of the vision, game logic and motion control runs on a single Raspberry Pi. The chess clock, prompts and messages appear on any phone or computer browser.

The key idea is that the camera is on the arm, so the robot can move to see better. When it is unsure what it is looking at, it views the board again from a different angle, and it uses the same approach to line up each grasp.

1.1 Features

- Plays with real Staunton pieces, with no magnets or electronics in the board
- 6DOF metal arm: five joints plus a two-finger gripper
- Wrist-mounted 12 MP autofocus camera
- Active vision: moves the camera to a better viewpoint whenever it is unsure
- Lines up each grasp visually before picking a piece
- Verifies every move, and retries or stops safely if something is wrong
- Stockfish engine with adjustable difficulty
- Supports captures, castling, en passant and promotion
- Chess clock and messages on a phone or computer
- Runs entirely on a Raspberry Pi Zero 2 W

1.2 Applications

- Over-the-board play against an engine
- Practice at an adjustable playing strength
- Demonstrating robotics and computer vision

1.3 Key Figures

Table 1. Typical performance

Placement accuracy	About ±3 mm
Ordinary move	About 25 s
Capture	About 40 s
Reach	All 64 squares and the capture tray

2 How a Move Is Played

Each turn follows the cycle in Figure 1. The robot never assumes a step succeeded. It checks the board before acting and again afterwards.

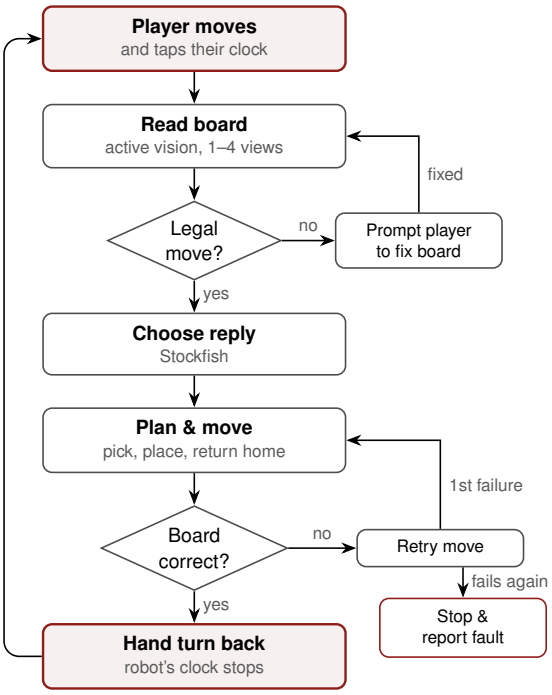


Figure 1. Turn cycle

3 System Architecture

The robot is built from off-the-shelf parts around a single Raspberry Pi Zero 2 W. Figure 2 shows how the hardware is connected. Figure 3 shows how the software turns a camera image into an arm movement and checks the result.

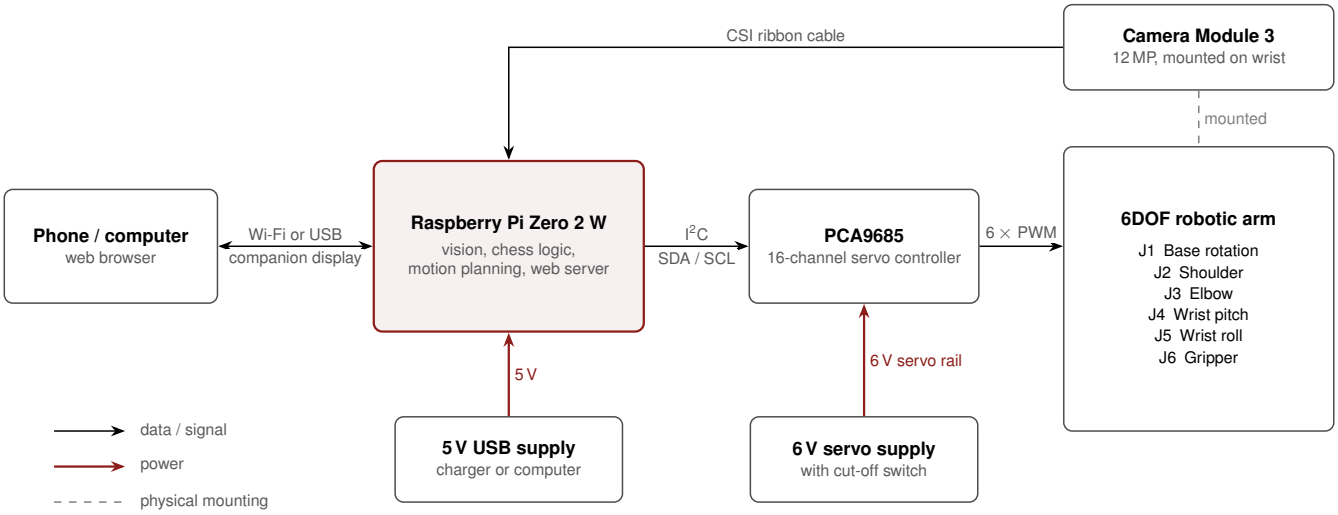


Figure 2. Hardware block diagram. The servos are powered separately from the Raspberry Pi, and all supplies share a common ground.

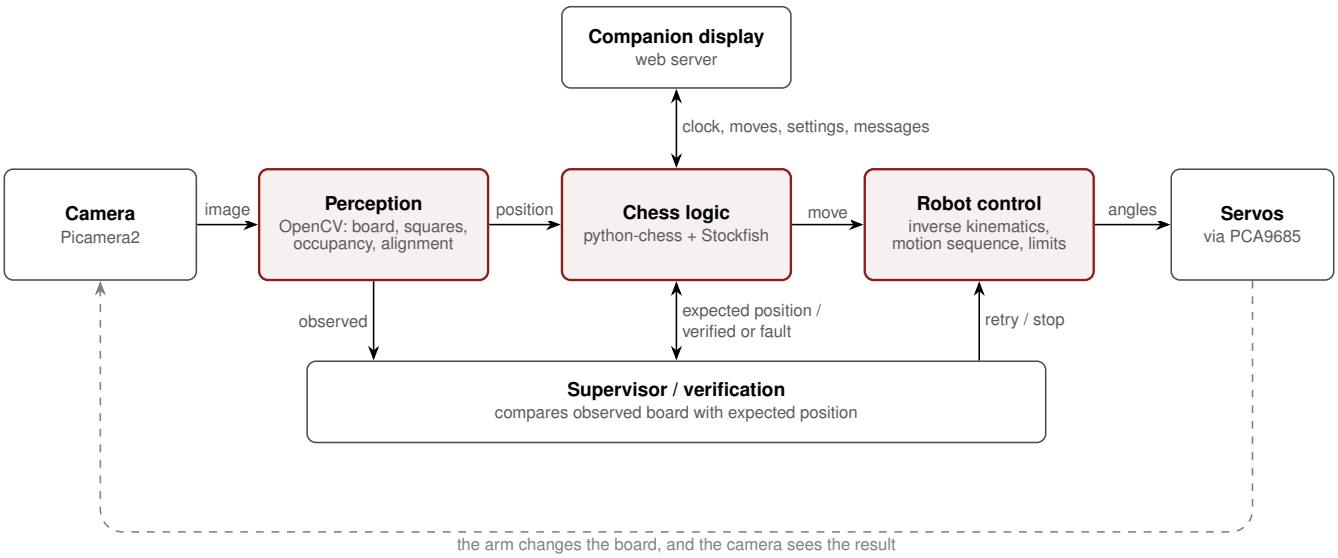


Figure 3. Software architecture. The supervisor checks every action against the expected position, closing the loop between the camera and the arm. Results and faults are passed back to the game, which shows them on the companion display.

3.1 The Four Subsystems

Perception. The wrist camera and computer-vision software locate the board from its markers, estimate how confident they are about each square, and locate pieces relative to the gripper.

Chess logic. The software keeps track of the game, checks the player's move and asks Stockfish for a reply.

Robot control. A move such as `e2e4` is converted into positions on the board, then into joint angles, then into a safe sequence of arm movements. The PCA9685 then drives the servos.

Verification. After every action, the camera checks the board against the expected position. If they don't match, the robot tries again or stops.

The Raspberry Pi runs all four. The PCA9685 only generates the servo signals. The servos take their power from a separate supply, because they draw far more current than the Pi can provide.

4 Feature Description

4.1 Active Vision

The camera rides on the arm, so the robot is not limited to one fixed view. It can look at any part of the board from any angle within reach. Rather than relying on one perfect image, it gathers evidence from several views and decides for itself when it has seen enough. The robot uses this *active vision* twice: to read the board, and to line up each grasp.

4.2 Reading the Board

Locating the board. Small printed markers around the edge of the board let the robot work out exactly where any view sits on the board, even when only part of the board is in frame.

Confidence per square. Each view gives every visible square a probability of being occupied. How much that view counts depends on how clearly it sees the square. A straight-down, close view counts for more than a low, oblique one. Squares that may be hidden behind a tall piece or affected by glare count for less.

Combining views. Each new view adds to the evidence instead of replacing it, so the estimate for every square improves as the robot looks.

Choosing where to look next. If some squares are still uncertain, the robot moves the camera to the view that sees those squares best. For example, it looks from the other side of a tall piece, or from higher up to reduce hiding (Figure 4).

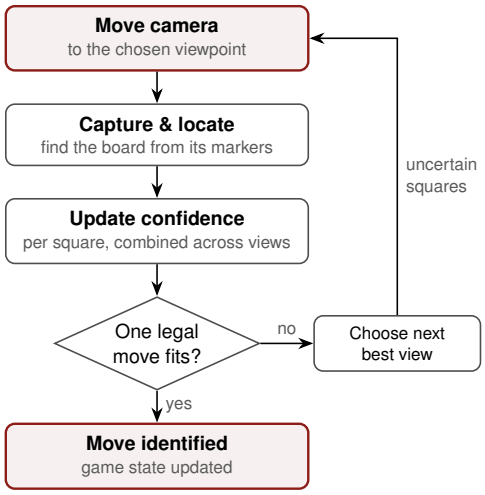


Figure 4. Active board reading

4.3 Detecting the Player's Move

The robot does not need to be certain about all 64 squares. It already knows the previous position, so the player can only have made one of about 20 to 40 legal moves. Each of those moves predicts a change to just two to four squares. The robot keeps looking only until the evidence picks out one of those moves clearly, and it chooses views that best tell the remaining candidates apart.

As a result, most turns need only one or two views, and difficult positions get more. The robot never has to recognise which piece is which, because the game history already tells it. If no legal move fits, or the position is still unclear after four views, the game pauses and the player is asked to check the board.

4.4 Choosing a Reply

The robot's reply is chosen by Stockfish, one of the strongest chess engines available. The difficulty can be set before each game, from beginner level, where the engine deliberately makes mistakes, up to full strength.

4.5 Motion Planning

The robot knows where the board, the camera, the arm base and the gripper are relative to each other. It uses this to convert a square on the board into joint angles for the arm. Every movement is checked against the arm's safe range before it is sent to the servos. Each pick-and-place follows the same eight-step sequence (Figure 5).

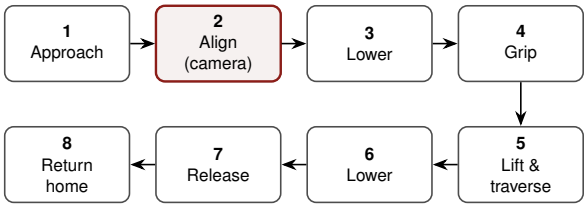


Figure 5. Pick-and-place sequence

4.6 Visual Grasp Alignment

Hobby servos have some play, so the arm arrives above a square close to the target but not exactly on it. The camera closes the gap. It measures how far the piece is from the centre of the gripper, the arm makes a small correction, and the camera looks again. This is closed-loop visual servoing. It normally converges to within 2 mm in one or two corrections. If it has not converged after three, the robot reports a fault instead of grabbing blindly.

Pieces are gripped near the top, where they are narrowest, so that the gripper fingers clear the pieces on neighbouring squares.

4.7 Verification and Recovery

After placing a piece, the robot reads the board again and compares it with the position it expects, looking closely at the squares involved in the move. If a piece is missing, misplaced or still in the gripper, it tries again. If the second attempt fails, the arm returns home, stops and reports the problem.

4.8 Companion Display

The robot has no screen of its own. Instead, it serves a simple web page that opens in the browser of any phone, tablet or computer, with nothing to install. It connects either over Wi-Fi, or through the USB cable when the Raspberry Pi is powered from a computer.

The companion display shows:

- both players' clocks: tap your side to end your turn
- game settings: difficulty, time control and the robot's colour

- the list of moves played
- prompts, for example *“Place a queen on e8”*
- errors, for example *“Illegal move: return the piece on e4”*
- faults, for example *“Could not pick up the piece on g8”*

4.9 Chess Clock

The clock times both players. The player’s clock stops when they tap it on the companion display. The robot’s clock stops once its move has been placed and verified. The game can also be played untimed.

5 Chess Rule Support

The chess engine makes sure every move is legal. The robot handles the physical side of each type of move.

Table 2. Physical handling of moves

Ordinary move	Picks up the piece and places it on its new square
Capture	Moves the captured piece to the capture tray first, then makes the move
Castling	Moves the king, then the rook
En passant	Removes the captured pawn, then moves the capturing pawn
Promotion	Removes the pawn; the player places the new piece when prompted
Illegal move	Pauses and shows which square needs correcting

6 Hardware

Table 3. Main components

Computer	Raspberry Pi Zero 2 W (512 MB RAM)
Camera	Raspberry Pi Camera Module 3, 12 MP, autofocus
Camera connection	CSI ribbon cable, 22-pin to 15-pin
Servo controller	PCA9685, 16-channel PWM, connected over I^2C
Robotic arm	6DOF metal arm: five joints plus gripper
Actuators	Six servo motors (rated 4.8 V–7.2 V)
Gripper	Two-finger claw
Storage	microSD card
Board markers	Printed ArUco markers around the board edge

7 Specifications

Typical values for the complete system. These will be updated with measured results as testing is completed.

Table 4. Physical

Footprint	420 × 260 mm (board and arm base)
Height	380 mm (arm in home position)
Weight	2.6 kg (without pieces)
Board	224 × 224 mm, 28 mm squares
Pieces	Staunton, base up to 20 mm, height up to 60 mm

Table 5. Arm and gripper

Joints	5 joints plus gripper (6 servos)
Maximum reach	400 mm, fully extended
Working reach	About 280 mm with the gripper pointing down
Payload	150 g at full reach
Gripper opening	0–55 mm
Joint speed limit	60 °/s (set in software)

Table 6. Electrical

Raspberry Pi supply	5 V USB, 2.5 A adapter
Servo supply	6 V regulated, rated 10 A
Servo current	About 2.5 A while moving
Peak servo current	Up to 7.5 A

Table 7. Vision

Board location	Printed markers around the board edge
Views per board read	1–4, typically 1–2
Board read time	About 3–10 s, depending on views
Grasp alignment	Up to 3 corrections, to within 2 mm
Lighting	Even, at least 300 lux

Table 8. Performance

Placement accuracy	±3 mm, with visual correction
Repeatability	±5 mm, without visual correction
Pick-and-place	98 % succeed first time
Engine reply	1–5 s, depending on difficulty
Ordinary move	About 25 s
Capture	About 40 s
Start-up	About 45 s to ready

8 Power

The Raspberry Pi and the servos are powered separately (Figure 2).

- **Raspberry Pi**: 5 V over USB, from a phone-style charger or a computer.
- **Servos**: a separate regulated 6 V supply connected to the servo controller's power rail. The servos draw large currents when they start moving or hold a load, so they must never be powered from the Raspberry Pi.
- **Ground**: the Raspberry Pi, servo controller and servo supply share a common ground.

The servo supply should never exceed 7.2 V.

9 Software

Table 9. Software components

Raspberry Pi OS	Operating system
Python	Main application
Picamera2	Camera control
OpenCV	Marker detection, square occupancy, piece location
Active vision	Confidence per square, choosing the next view
python-chess	Game state and chess rules
Stockfish	Move selection
Robot control	Arm kinematics and motion sequences
Web server	Companion display

The vision software uses standard image-processing techniques rather than large AI models, which keeps it light enough to run on the Pi Zero 2 W.

10 Setup

1. Place the arm and board on a flat table, with the arm base centred along one edge of the board.
2. Set up the pieces and put the capture tray in place.
3. Power the Raspberry Pi, then switch on the servo supply.
4. Open the companion display on a phone or computer.
5. On first use, run calibration from the companion display.
6. Choose the difficulty, time control and colour, then start the game.

Calibration measures the offset between the camera and the gripper, and records the arm position above each square. Because the board markers let the robot locate the board in any view, small knocks to the board are corrected automatically. Full calibration is only needed if the board is moved to a new position. The board should be evenly lit, without strong shadows or glare.

11 Safety

- Keep hands away from the arm while it is moving.
- The servo supply has a power switch within easy reach. Switch it off before reaching into the arm's workspace.
- Arm speed and joint range are limited in software.
- The arm returns to a safe home position after every move.
- If anything goes wrong, the arm stops and the problem is shown on the companion display.

12 Design History

12.1 Version 1: Magnetic Board

MagneticTurk began as a self-playing magnetic chessboard based on the open-source ChessBot V1 design. A belt-driven CoreXY mechanism under the board moved an electromagnet that dragged pieces with metal bases across the surface. An ESP32 read a grid of 64 Hall-effect sensors to detect which squares were occupied. A custom circuit board for the sensors, motor drivers and magnet driver was designed and manufactured, and the frame was modelled for 3D printing.

12.2 Version 2: Robotic Arm

Dragging pieces from below has limits: pieces have to slide between other pieces, captures are awkward, and the sensors can tell that a square is occupied but not which piece is on it. The design was therefore changed to a robotic arm that lifts pieces from above, with a camera to guide it. As a result, the robot no longer needs a special board or magnetic pieces.

12.3 Hall-Sensor Board

The original Hall-sensor board was considered as a second way of checking moves, mounted underneath the playing surface. The camera proved reliable enough on its own, and dropping the sensor board removed a second controller, extra wiring and an extra calibration step.

12.4 Development Approach

The arm version was built in stages, so that each part could be tested before the next was added: assembling the arm, setting up the Pi and camera, detecting the board, testing the servos one at a time, positioning the arm, grasping a piece, full pick-and-place, connecting the chess engine, and finally verification and special moves.

13 Acknowledgements

Version 1 is based on ChessBot V1 by Joshua Stanley Robotics (MIT License). Stockfish is free software licensed under GPLv3.

Use of AI. This is a personal project, and AI tools (Anthropic's Claude) were used throughout as a design and writing assistant. They helped plan the system architecture, review the design for feasibility, and draft and typeset this document. The design decisions, hardware build and testing are my own.